Does an LLM Reorder Investment Candidates Better Than LambdaMART? A Same-Information Ranker Study on FAR-Trans

Student: Daria Babikova (Бабикова Дарья)
Team: Team 5 — Alina Kozhokar, Uliana Tselishcheva, Daria Babikova
Date: 2026-11-13 · Assignment: Session 1 — Research Question and Minimal Study Plan (Define) · Track: Ranker · State: planned
Repository: https://github.com/dbabikova/RecSys-LLMs/tree/main/project_group_2module (commit f4803f4)

Abstract

Problem. The course task asks for a Top-10 list of financial assets per FAR-Trans customer [1], scored by nDCG@10 against future purchases, with repeat purchases allowed and relevant. LLM rankers can reorder candidate lists [3], but on financial data the only LLM study we found [2] reports an unranked top-3 hit rate, no nDCG@10 and no same-information control. **Method.** I ask whether zero-shot LLM reranking of a fixed top-20 candidate list beats a LightGBM LambdaMART reranker that sees the same fields and the same candidates, and plan the comparison inside the course starter [8] with its four baselines, four ablations, runtime and cost. **Result.** No model has been run. The starter baselines reproduce exactly: on the full warm cohort (10,404 users, 1,178 with purchases) Latest buy reaches nDCG@10 = 0.4648 on users with purchases, CBF 0.2522, Popularity 0.2312. A ceiling check shows that even a perfect reordering of the starter's CBF top-50 reaches only 0.4544, because 61% of future purchases are repeats that CBF often omits. **Takeaway.** The question is testable and isolates the method from the information; its first Session-2 step is a repeat-aware candidate pool shared by both rerankers.

Index Terms: financial asset recommendation, FAR-Trans, LLM reranking, LambdaMART, repeat purchases, nDCG@10, same-information control, leakage.

1. Introduction

Problem statement. FAR-Trans [1], [7] holds 388,048 transactions of 29,090 retail customers of a Greek bank (2018-01 … 2022-11) over 806 stocks, bonds and mutual funds, plus customer profile snapshots and daily prices. The shared task is one Top-10 list per evaluated customer, scored with nDCG@10 against assets bought in a future window. On 13 Nov 2026 the instructor confirmed that previously purchased assets may be recommended and count as relevant, unlike the new-asset-only protocol of [1], and asked teams to keep the starter's repeat-allowed default and compare with its four baselines. The final cut, users and asset scope are still pending, so I use the starter defaults as the working protocol: cut 2021-06-01, a 30-day label window, the deterministic 100-user demo cohort, plus the full warm cohort.

Research question. *We test whether* **zero-shot LLM reranking of a fixed top-20 candidate list** *improves* **nDCG@10** *over* **a classical learned reranker (LightGBM LambdaMART) that sees the same user and asset information and the same candidates** *for* **the common FAR-Trans evaluation users.** Intervention: a frozen, prompted open-weight LLM orders the 20 candidates. Control: LambdaMART on the same candidates and fields. Outcome: nDCG@10 (runtime and cost secondary). Population: the common evaluation users, with pre-specified subgroups (repeat vs. new target purchases; history length). Track: Ranker; state: planned. No direction is assumed; a null or negative result will be reported.

Concrete example. One demo customer (Professional, risk *Aggressive*, observed) bought National Bank of Greece three times in 2020, sold it in July and September 2020, and bought it again on 2021-06-08, inside the label window. The starter's CBF ranks it 10th (eight food-sector stocks tie ahead of it on identical attributes), giving this customer nDCG@10 = $1/\log_2 11$ = 0.289; Latest buy ranks it 5th, giving $1/\log_2 6$ = 0.387. Moving it to rank 1 would give 1.0. Reordering a fixed list is exactly the decision the reranker makes, and recognising a return to a sold asset needs the history, which an LLM can read as text.

Research importance. LLM rankers suffer position and popularity bias and read history order poorly [3]; in a matched candidate pool, no open-weight LLM reranker beat a tuned shallow collaborative baseline, and the candidate generator drove much of the measured gain [4]. Financial assets add thin, partly Greek metadata and strongly repetitive demand, where simple frequency and recency rules are hard to beat [5]. Whether an LLM still adds ordering value over a learned ranker that uses the same fields is untested on this data; a positive and a negative answer are both new knowledge.

Business value. The decision that changes is the product owner's choice of ranking layer for an advisory recommendation rail: pay for GPU inference of an LLM ranker, or keep a CPU LambdaMART or a plain recency rule. Investors are poor judges of advice quality in a user study with FAR-Trans authors [9], so the choice should rest on an offline comparison against real behaviour at equal information. No claim on returns, suitability or user acceptance is made: nDCG@10 measures ranking accuracy against observed purchases only.

Contributions.

- One RQ with a same-information LambdaMART control and ablations that separate asset knowledge, personalisation, profile and position bias (§3).
- A leakage audit of every input field, including two non-obvious future-information sources (§3.1).
- An exact reproduction of the four starter baselines and a candidate-ceiling finding that fixes the first step of Session 2 (§4).

2. Related Work

Study	Method	Data and evaluation	Limitations and connection to my RQ
Sanz-Cruzado et al., 2024, Fin-RecSys@IJCAI [1]	11 FAR algorithms: price-based regressors, Popularity, LightGCN, MF, kNN, ARM, hybrids incl. LambdaMART	FAR-Trans; 61 cuts, 6-month test; Rel = buy; repeat pairs dropped; nDCG@10, ROI@10	No LLM; new-asset-only labels differ from the course rule. Source of data and of LambdaMART as an existing learned ranker
Spadea & Seneviratne, 2025, ICAIF [2]	FLARKO: Qwen3 0.6–8B, LoRA + KTO, transactions and market as knowledge graphs; LLM outputs ISINs	FAR-Trans; test 2021-12 … 2022-06, 180-day labels; Pref@3 / Prof@3 / Comb@3	Nearest prior work. Unranked top-3, no nDCG@10, no fixed-pool reranking, no same-information classical control
Hou et al., 2024, ECIR [3]	Zero-shot LLM ranks retrieved candidates from history; recency prompts, candidate permutations	ML-1M, Amazon Games; 20 candidates; nDCG@K	Position and popularity bias; weak use of history order. Motivates permutations and the no-history ablation
Kapetanovic et al., 2026, arXiv [4]	Retrieve-then-rerank with proprietary, open-weight and LoRA LLM rerankers vs. CF/sequential baselines	ReDial; matched pools 250–full; nDCG@10, Hit@10	Open-weight rerankers lose to a tuned shallow CF; pool source changes results by >50%. Same pool ≠ same information
Li et al., 2023, ACM TOIS [5]	Next-basket reality check: frequency/repeat baselines (GP-TopFreq) vs. deep models; repeat vs. explore analysis	TaFeng, Dunnhumby, Instacart; Recall/nDCG@10,20	Gains of complex models often come from repeats. Explains why Latest buy is the bar to beat under the course rule

Table 1. Related studies used for the RQ (full texts opened). Method = what the model does; Data and evaluation = datasets and metrics as reported; last column = gap or design consequence for this study.

Nearest prior work and gap. FLARKO [2] is the closest LLM study on FAR-Trans. It answered whether a small fine-tuned LLM with graph context can name assets a customer buys and that are profitable (top-3 hit rate). Under my condition it left open the Top-10 ranking quality in nDCG@10 with repeat purchases relevant, and whether any gain comes from the LLM or from the extra information it receives. A non-LLM reranker on FAR-Trans exists [6], so reranking itself is not new here. *We did not find a study reporting nDCG@10 for zero-shot LLM reranking of a fixed candidate pool on FAR-Trans against a non-LLM learned reranker with the same input fields, with repeat and new purchases reported separately and with runtime and cost.*

Alternatives considered. (a) *Encoder* — frozen LLM encoding of the purchase profile vs. a structured encoding with user-kNN fixed (Alina's proposal, §5). (b) *Hybrid* — retrieval change plus reranking in four configurations (Uliana's proposal, §5). (c) LLM vs. the original candidate order only — rejected as the main comparison because it confounds method with information; the original order stays as a reference.

3. Method

3.1 Data, one real record and leakage

Source. FAR-Trans archive [7] (CC BY 4.0, SHA-256 d0d6cfe0…cfb6), processed by the unmodified course starter [8] (commit 5133703). Starter rules: history = Buy rows strictly before the cut; reconstructed opening positions (negative transactionID, 38,019 rows) and exact duplicates excluded; an asset is eligible if it has as-of metadata and a positive pre-cut price; truth = Buy in [2021-06-01, 2021-07-01) intersected with eligible assets; users with no purchase in the window stay in the mean with nDCG = 0. Item scope: all asset classes allowed by the starter; a stocks-only scope is not assumed. One record from the label window, customerID masked, other values verbatim:

```
source:    FAR-Trans public release (researchdata.gla.ac.uk/1658), transactions.csv
record id: transactionID 9049523 (non-identifying; customerID masked)
fields:    customerID, ISIN, transactionID, transactionType, timestamp, totalValue, units, channel,
marketID
values:    [MASKED], GRS003003035, 9049523, Buy, 2021-06-08, 996.41, 370.0, Phone Banking, XATH
window:    label window 2021-06-01 <= timestamp < 2021-07-01 (starter demo) — the target, never an
input
```

Field (table)	Used by	Status	Timestamp in the example	Leakage rule
this record: ISIN, transactionType, timestamp	label only	observed (date may be shifted by the release)	2021-06-08 (≥ cut)	never an input
this record: totalValue, units, channel, marketID	not used	totalValue estimated (units × close price); others observed	2021-06-08	known only at purchase → leakage
past Buy: ISIN, buy_count	LLM + LambdaMART	observed; Sell rows and the reconstructed 2018 buy excluded	3 buys 2020-03-27 … 2020-04-06	only rows < cut
customerType	both	observed	snapshot 2021-03-10	latest snapshot < cut
riskLevel / investmentCapacity	both	observed here; Predicted_* for ~25% of latest profiles	snapshot 2021-03-10	< cut; withheld if questionnaire ≥ cut

Field (table)	Used by	Status	Timestamp in the example	Leakage rule
lastQuestionnaireDate	availability check only	observed; 2000-01-01 = placeholder	2018-12-10	use only if < cut
asset name, category, sub-category, sector, industry, market	both	observed; sector and industry missing for this asset	asset row 2018-01-02	static; historical names unproven
limit_prices.profitability, prices ≥ cut	never	derived over the whole period	to 2022-11-29	excluded

Table 2. Input fields for the example customer. Status: observed / predicted (estimated) / missing as defined by the release; leakage rule = condition for use at prediction time (cut 2021-06-01).

Two less obvious points: the starter context gives a purchase count but no purchase dates, so recency features would have to be added symmetrically to both rerankers; and profile snapshots are not strictly point-in-time (in the 2022 audit 1,209 as-of snapshots carried a questionnaire dated after the cut), so risk and capacity are withheld from both methods whenever the questionnaire is not earlier than the cut.

3.2 Arms, same information and the fixed candidate set

Arms. The four starter baselines — Popularity, CBF, Latest buy, Latest buy or sell — are kept unchanged on their full eligible universe; the original pool order is a reference; **C** LightGBM LambdaMART (objective lambdarank, group = user × cut) is the main control; **L** the LLM reranker is the intervention. **Same information.** C and L receive one payload: history assets with names, attributes and buy counts, the as-of profile, candidate names and attributes, and the generator's score and rank. C gets them as features (categorical encodings, history–candidate attribute overlaps, TF-IDF of names fitted on training data only), L as text. C is trained on earlier cuts whose label windows end before 2021-06-01, with 5 seeds. **Candidates.** A fixed top-20 pool, identical for C and L, isolates reranking; the starter's CBF is the default generator. Because CBF misses many repeat purchases (§4.2), Session 2 first fixes a repeat-aware pool built only from pre-cut information, then freezes it for both arms. **Shared output.** Every arm emits the common Top-10 for the same users, scored by the starter's nDCG@10 code.

LLM ranker (proposal, frozen in Session 2). An open-weight instruction model run locally — candidate `Qwen/Qwen3-8B`, exact Hugging Face revision reported — zero-shot, temperature 0, answering in the starter's ranker format (ten unique IDs from the pool). Against position bias [3], each user is scored under 3 fixed candidate permutations (seeds 17, 42, 2026) and the three Top-10s are merged by rank points. An invalid answer falls back to the pool's original Top-10, is flagged and counted, and the user stays in the metric. No LLM judge is used; the only ground truth is purchases.

3.3 Measures, contribution check and risks

Measures. Primary: nDCG@10 averaged over all selected users (empty truth = 0); also the mean over users with purchases and their number, as the starter reports. Paired per-user difference L − C; spread over the 3 permutations and over the 5 LambdaMART seeds. Secondary: latency per user (mean, median, p95) and full-run time on the same machine, GPU-hours, prompt tokens, invalid outputs, and the share of popular assets in the lists. Subgroups: repeat vs. new target purchases; retained history length 1–5, 6–20, > 20 buys.

Ablation (applied to both C and L where it removes information)	What it isolates
No asset names (aliases instead of names and ISINs, attributes kept)	issuer knowledge and possible hindsight from pretraining
No explicit history (history and history-based scores removed)	personalisation vs. a popularity prior inside the LLM
No profile (customer type, risk, capacity removed)	contribution of the profile, including estimated values
Original pool order, no permutations (L only)	sensitivity to presentation order at fixed content

Table 3. Ablations; candidates, users, labels and metric stay unchanged; when a field is removed, LambdaMART is retrained without it.

Risks. *Contamination:* the LLM was pretrained after 2021 and may know what happened to Greek assets afterwards; a gain that disappears without names is read as possible hindsight, not skill. *Privacy:* pseudonymised data; customer IDs replaced by aliases and never placed in a prompt; local inference only. *Ethics:* ranking accuracy is not suitability or profit; no advice is given. *Data:* retrospective reconstruction of opening positions, shifted dates and adjusted prices in the release; tied CBF scores on identical metadata.

Feasibility. Prompts ≈ 1k tokens for 20 candidates; budget up to 8 GPU-hours and one CPU-day, no paid model API; GPU availability checked in Session 2. Session 2: fix the repeat-aware pool, LambdaMART, actual LLM inference with parser and fallback, pilot runtime, demo comparison. Session 3: full controlled comparison and first ablation. Session 4: remaining ablations, error analysis. Sessions 5–6: integration with the team design, reproducible final run, defence. Tools: Python, starter evaluator, LightGBM, Transformers or vLLM; commits, CSV hashes, prompts and model revision recorded.

4. Experiments

No reranker has been run; §4.3 is the planned evaluation. §4.1–4.2 report checks produced by committed scripts in the repository.

4.1 Data accounting and starter baselines

The data audit matches the dataset paper [1]: 388,048 transactions, 29,090 customers, 806 assets, 228,913 Buy rows, of which 38,019 are reconstructed opening positions; 68.7% of real buys repeat an asset the customer bought before; the ten most bought assets take 48.2% of Buy rows; sector is missing for 47.6% of assets.

Cohort	Users / with purchases	Popularity	CBF	Latest buy	Latest buy or sell	Run time
Demo (starter default)	100 / 11	0.0286 / 0.2599	0.0201 / 0.1827	0.0277 / 0.2520	0.0271 / 0.2464	7.7 s
Full warm	10,404 / 1,178	0.0262 / 0.2312	0.0286 / 0.2522	0.0526 / 0.4648	0.0509 / 0.4496	47.7 s

Table 4. Starter baselines, nDCG@10 as all users / users with purchases; cut 2021-06-01, 30-day window, repeats relevant. Both rows reproduce the starter README to four decimals. Run time is for the whole CLI run, not per method.

4.2 Candidate ceiling

On the full warm cohort, 61% of future purchase pairs are repeats. CBF top-20 holds 41% of a user's future purchases on average (55% of repeats, 13% of new assets). A perfect reordering of CBF top-50 would reach nDCG@10 = 0.4544 on users with purchases — below Latest buy (0.4648). A reranker of the CBF pool therefore cannot beat the strongest baseline even with perfect ordering; the pool, not the ranker, limits the result, as [4] observed on movies. This is why the shared pool must include the user's own past assets. On the 11-user demo the ceiling is not binding, which also shows that demo numbers alone cannot support conclusions.

4.3 Planned evaluation

All arms on the frozen protocol and pool; nDCG@10 overall, on users with purchases and per subgroup; pool recall as ceiling; runtime, cost, invalid outputs; then the four ablations. Reading: L > C — the LLM adds ordering value beyond a learned ranker with the same fields (checked against hindsight by the no-name ablation); L ≈ C — no measurable gain at this metadata richness, a useful negative result for the GPU-vs-CPU decision; L < C — zero-shot ranking is hurt by bias in this domain, which motivates tuning in Session 4. Either way the four starter baselines are reported next to C and L.

4.4 Verification

- **My own check.** In the raw CSV files I located transaction 9049523 (one row) and confirmed its values; found the customer's three earlier real purchases of the same asset (2020-03-27, 2020-04-01, 2020-04-06), a reconstructed 2018 buy with negative ID and two sales in 2020, so the target is a repeat purchase after selling; confirmed the as-of profile snapshot of 2021-03-10 (Professional, Aggressive, CAP_80K_300K, questionnaire 2018-12-10) and the asset row (National Bank of Greece, Stock, Large Cap, no sector or industry). I computed nDCG@10 by hand: $1/\log_2 11 = 0.2891$ (CBF, rank 10) and $1/\log_2 6 = 0.3869$ (Latest buy, rank 5). The ranks were taken from the script output; I did not recompute them.
- **Supporting re-runs (not my own checks).** The AI-written `scripts/verify_example.py` recomputes the example from the raw CSVs without the starter code: 46 / 46 checks match. The starter's own 58 tests pass, and the baseline tables match the starter README to four decimals.

5. Discussion

Team discussion. We discussed the advantages and limitations of three proposals. Alina proposed changing the purchase-profile encoding while keeping user-kNN fixed (Encoder). I proposed LLM reranking against LambdaMART on a fixed top-20 list (Ranker). Uliana favoured a Hybrid design combining asset-metadata retrieval and reranking, with four configurations separating the two contributions. We agreed to revisit the choice in the next assignment; the shared design and the division of work remain open.

Agreement and unresolved differences. Converged: one shared FAR-Trans task, the starter protocol and the instructor's repeat-allowed rule; each proposal changes one stage under equal information. Unresolved: whether asset names and profile fields enter the model (Alina's plan excludes them; mine includes them and measures them by ablation), and which candidate pool a reranker should receive.

My contribution. I proposed the Ranker question and the same-information LambdaMART control, directed the data audit, the starter reproduction and the candidate-ceiling check, and brought the ceiling result to the team: CBF top-50 cannot exceed Latest buy, so any reranker — mine or the ranking stage of a Hybrid — depends on a repeat-aware pool. I personally verified the data example (§4.4). Shared: the task framing and the starter.

My position and reasons. I adopt the Ranker question. It changes one measurable thing — the order of a fixed list — while the information is held equal, so a difference is attributable to the method; and the gap to [2] is concrete. The ceiling finding supports Uliana's point that a reranker only works with the candidates it receives; I therefore make the shared repeat-aware pool the first step of Session 2, which is also the retrieval half of her Hybrid. Alina's Encoder tests the other stage; I see the three proposals as parts of one Hybrid rather than rivals.

Threats to validity. The starter teaching protocol may differ from the final one; the demo cohort has only 11 users with purchases; metadata are thin and partly Greek; demand is concentrated; the LLM may carry hindsight knowledge; the release reconstructs and shifts some events; profile snapshots are not strictly point-in-time.

6. AI Usage Disclosure

AI tools used. (1) *Coding agent:* OpenCode, main model OpenAI `gpt-6.1-sol` (session ses_e2b3ab5f…; at the time of writing 13 user messages including resends, 170 assistant messages, 294 tool calls: 127 shell, 69 read, 37 web fetch, 35 patch, 18 web search; recorded cost $18.35). Two first attempts on free models (big-pickle, mimo-v2.6-flash-free) stopped at the provider limit; one of them read README and DECISIONS of my earlier folder `project_module2`, which were not carried into this work. The agent wrote and ran all scripts in `project_group_2module/`, searched and read the literature,

reproduced the starter and drafted the reports, committing under my Git identity. (2) *Study assistant:* Claude (Anthropic, Claude Opus 5.5) in a separate chat wrote my prompts for OpenCode, checked each step's outputs against the repository, read my teammates' reports for §5, and drafted and typeset this report.

How AI was used and what I decided. I chose the Ranker track, the research question and the LambdaMART control, directed each step, asked the instructor-rule question to be recorded, and decided to keep pool selection for Session 2.

What I personally verified. The data example in the raw CSVs and the hand nDCG@10 (§4.4).

What I trusted without full verification. All dataset counts and baseline tables (script output; the baselines match the starter README); the summaries of the related studies; the candidate-ceiling computation; the budget estimate.

Session log. `babikova_session1_session.json` is the unmodified native OpenCode export of the session above. The Claude chat has no native export with tool results; a derived text export without code or tool outputs can be provided on request.

References

[1] J. Sanz-Cruzado, N. Droukas, and R. McCreadie, "FAR-Trans: An investment dataset for financial asset recommendation," in Proc. IJCAI-2024 Workshop on Recommender Systems in Finance (Fin-RecSys), 2024. [Online]. Available: https://arxiv.org/abs/2407.08692

[2] F. Spadea and O. Seneviratne, "Aligning language models with investor and market behavior for financial recommendations," in Proc. 6th ACM Int. Conf. AI in Finance (ICAIF), 2025, doi: 10.1145/3768292.3770399. [Online]. Available: https://arxiv.org/abs/2510.15993

[3] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. McAuley, and W. X. Zhao, "Large language models are zero-shot rankers for recommender systems," in Advances in Information Retrieval (ECIR 2024), 2024, pp. 364–381, doi: 10.1007/978-3-031-56060-6_24.

[4] A. Kapetanovic, T. Duricic, A. Mercep, and E. Lacic, "Retrieval, scoring, and decoding shape performance and stability in LLM-based conversational recommendation," arXiv:2609.00086, 2026, doi: 10.1145/3799682.3840066. [Online]. Available: https://arxiv.org/abs/2609.00086

[5] M. Li, S. Jullien, M. Ariannezhad, and M. de Rijke, "A next basket recommendation reality check," ACM Trans. Inf. Syst., vol. 41, no. 4, pp. 1–29, 2023, doi: 10.1145/3587153.

[6] R. Unno, K. Sakurai, K. Watanabe, K. Maeda, T. Ogawa, and M. Haseyama, "One basket, many experts: Decorrelated re-ranking for financial asset recommendation," accepted to ICAIF 2026. [Online]. Available: https://arxiv.org/abs/2611.05952

[7] University of Glasgow, "FAR-Trans dataset," research data repository, 2024, CC BY 4.0. [Online]. Available: https://researchdata.gla.ac.uk/1658/

[8] S. Jin, "FAR-Trans student starter," LLM4Rec course repository, commit 5133703, 2026. [Online]. Available: https://github.com/dryjins/RecSys-LLMs/tree/main/project/far-trans

[9] T. Takayanagi, K. Izumi, J. Sanz-Cruzado, R. McCreadie, and I. Ounis, "Are generative AI agents effective personalized financial advisors?," in Proc. SIGIR 2025, pp. 286–295, doi: 10.1145/3726302.3729897.

[10] LLM4RecSys course, "Lecture 07. LLM-Based Recommendation and FAR-Trans Project," HSE University, 2026 (course slides).